

On the beating of horses, and other matters

Document #: P3613R0
Date: 2025-02-15
Project: Programming Language C++
Audience: SG9
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

*quo usque tandem abutere, Catilina, patientia nostra?
quam diu etiam furor iste tuus nos eludet?
quem ad finem sese effrenata iactabit audacia?*

— Cicero, *In Catilinam I*

§ 1 Introduction

We should not beat horses. Dead or otherwise.

But sometimes a horse is in need of a course correction and need some gentle or not-so-gentle persuasion. The best time to do that is when the horse is still alive.

This is C++ though, where we have time travel and can DR a seemingly dead horse back to life. So the second-best time to do that is when it is just dead, as shown in [this photo](#) taken in the late 19th century.

We have, in fact, done this for the ranges library, soon after we shipped it in C++20. See, e.g., [\[P2017R1\]](#) [\[P2210R2\]](#) [\[P2325R3\]](#) [\[P2328R1\]](#) [\[P2415R2\]](#) [\[P2432R1\]](#). And we have continued to patch up minor issues since then, see, e.g., [\[P2602R2\]](#) [\[P2609R3\]](#) [\[P2997R1\]](#).

What we should not do, though, is descending upon that same crossing (which now looks [like this](#)), more than a century later, in search of that horse so that we can give it another good beating.

Yet it appears [\[P3329R0\]](#) wants to do just that. This paper is an attempt¹ to explain that the horse has already been thoroughly beaten and we should [drop our sticks and back slowly away from the imaginary horse carcass](#).

§ 2 `filter` and mutating elements

Let's start from some basics.

One of the defining characteristics – perhaps *the* defining characteristic – of forward iterators is that they are multi-pass.

What does multi-pass mean? 24.3.4.11 [\[iterator.concept.forward\]](#) p4 explains:

- 4 Two dereferenceable iterators `a` and `b` of type `X` offer the *multi-pass guarantee* if
- (4.1) — `a == b` implies `++a == ++b` and
- (4.2) — the expression `((void)[])(X x){++x;}(a), *a` is equivalent to the expression `*a`.

In human-readable terms, this means that you can iterate through the range multiple times and get the same things back each time.

This is not new to the C++20 ranges design. 24.3.5.5 [\[forward.iterators\]](#), which specifies the pre-C++20 forward iterators and has its origins in the original STL, has essentially the same definition.

Can *any* lazy filter range – regardless of the implementation – possibly meet that requirement if elements are mutated in a way that changes what the predicate returns?

No. A lazy range must defer the evaluation of the predicate until the point of iteration, so it can't preemptively *cache* (dare I say that word?) which elements are in the range. It has to evaluate the predicate when it gets there, and so if the element has been mutated so as to change the result of the predicate, then iterating through the range twice will produce different answers.

If I iterate through a range once and see four elements, and then I iterate through it again and only see two, then, by definition, that is not a multi-pass range.

A lazy filter view cannot both be forward and yet allow arbitrary mutation. *That is fundamental to the nature of the algorithm.* As has been pointed out many times.

So what can we do? Well, there are not a lot of options.

- We can say that the view is input and not provide the multi-pass guarantee. That disallows whole families of algorithms.
- We can disallow mutations altogether – say, by effectively adding a `views::as_const`. That disallows a variety of entirely sound code and [isn't bullet-proof either](#).
- We can allow mutations in general, but disallow it – on pain of undefined behavior – if that would change the result of the predicate and cause us to fail to provide the multipass guarantee despite advertising so.

The standard picked the last option. It makes `filter` as powerful as it can possibly be, within the fundamental constraints imposed by the definition of forward iterators. Implementations can opt to add debug checks that catch a good fraction of misuses, too – though it won't be all of them.

Now that we have `views::as_const` and will soon have `views::to_input`, it's trivial to build the first two options from the third, if so desired. It is hard – if not impossible – to build the last option from the remainder.

§ 3 Mutating the underlying range

View adaptors are lazy algorithms.

— Eric Niebler (slightly paraphrased)

A number of allegedly problematic examples in [P3329R0] involve various forms of mutating the underlying container while a `filter_view` refers to it and attempting to use the view afterwards.

Once we realize that range adaptors are lazy algorithms, the problem with such examples becomes obvious: they are equivalent to mutating a container while iterating over it. That’s generally a bad idea when the algorithm is eager. It’s not made less bad just because the algorithm is lazy.

Formally, such mutations are forbidden by the requirement that `ranges::begin` must be equality-preserving, non-modifying², and stable when applied to forward (or stronger) ranges – as long as the `filter_view` exists, messing with the underlying container is a violation of the semantic requirements on range, caching or not. And it’s not like these requirements are invented for no reason – they are critical to the ability to reason about generic code.³ But why let that stop the paper?

§ 4 **const-iteration and deep const**

View adaptors are lazy algorithms, and some algorithms have mutable state.

— Eric Niebler (slightly paraphrased)

As has been repeatedly explained, there is a wide variety of views that *must* maintain mutable state somewhere. `generator`. `istream_view`. `join` when joining a range of prvalues or an input range. `chunk` when chunking an input range. The list goes on.

Unless we are prepared to remove them all – and reject all future adaptors like them – an arbitrary range will never be uniformly `const-iterable`.

Nor will it be uniformly `deep-const`. That ship sailed with `boost::iterator_range`, a long time ago. And for copyable views in particular, `deep-const` is illusory anyway.

Cf. [P3431R0].

§ 5 **Why doesn’t filter “cache end”?**

Well, it has no need to. Its `end` is a wrapper around the underlying range’s `end`. In a lot of cases you can’t do better anyway: if the range isn’t bidirectional, or isn’t common, you can’t go back from the end, so short of walking through the whole range there’s not much that can be done.

But [P3329R0]’s “cache end” is something else: it suggests caching the position in the original range that is immediately past the last element. Why not do that, it asks triumphantly, if you care about performance?

Because often it is not necessary to iterate over every element in a range. And when it's not, calculating that position is a total waste of time. That's why not every algorithm in the standard library returns the end of its source range: often it's not necessary to reach the end.

§ 6 Complexity requirement of begin

The greatest mischief of [P3329R0], however, is not its proposal to “heal”⁴ `filter_view`, but its cavalierly proposed modification to the semantic requirements on `range`. Some might call it...audacious.

The `range` concept requires `begin` and `end` to be amortized constant time. Here, “amortized” means that every call after the first is constant. In cases where `begin` or `end` cannot be computed in constant time (like `filter`'s `begin`), then, this requires caching the position.⁵ The reason behind this design decision – which had been [debated at length over the span of several years](#) prior to standardization – is that it is not uncommon for range adaptors, and generic code in general, to call `begin` and `end` repeatedly, on the assumption that such calls are cheap. Having such calls perform a linear-time search each time can potentially turn a linear time algorithm into a quadratic one.⁶

Unfortunately, this design decision attracted the attention of [P3329R0] like the red cape in a bullfight, and its proposed change – allowing linear-time for `begin` – is fittingly not unlike a bull in a china shop.

The paper doesn't even attempt to discuss the extent of breakage caused by this change. What breaks? Well, everything. The entire programming model changes. User code that had assumed that `begin` can be called multiple times cheaply can now silently incur a significant cost – and has to do its own caching (which is actually tricky to do correctly) to get back the performance it lost.⁷ But we don't even need to go that far – there are plenty of examples in the standard library. I'm not going to attempt to do a comprehensive survey (that's the paper author's responsibility), but consider, for instance:

- `ranges::size`, which can compute `end()` - `begin()` in some cases.
- `view_interface::operator[]`, which does `begin()[n]`.
- `join_view::iterator::operator--`, which needs to know whether it's at the beginning of the current range.

And what's the deployment experience of this change to the most fundamental concept of the `ranges` library? The paper doesn't say. Surely not just the github repo with a small handful of the simplest views and a paltry 64 stars⁸, despite the author's attempts to promote it at multiple conferences over several years?

§ 7 A postscript

Time is a flat circle. Everything we have ever done or will do, we are going to do over and over and over again – forever.

— Rust Cohle, in *True Detective*

It is hard to believe that we are still relitigating design decisions embodied in the most fundamental concept of the entire ranges library *five years after* we shipped it in an international standard. This design space was exhaustively explored by the original authors of the ranges library before we standardized it, and again after that in the various forums of WG21.⁹ There is no new information coming in that would justify such an exercise.

Yet here we are.

On the basis of a paper whose exploration of the consequences of its proposed changes is so inadequate that calling it unbaked would be an insult to the ready-to-bake products available in supermarkets everywhere.

It is, of course, the chair's prerogative to choose which paper is discussed. And perhaps a re-re-exploration will miraculously find a unicorn where the horse was. If so, I would be pleasantly surprised. Until the unicorn materializes, however, I plan to spend my time in subgroups where it can be put to more valuable use.

§ 8 References

[P2017R1] Barry Revzin. 2020-02-19. Conditionally borrowed ranges.

<https://wg21.link/p2017r1>

[P2210R2] Barry Revzin. 2021-03-05. Superior String Splitting.

<https://wg21.link/p2210r2>

[P2325R3] Barry Revzin. 2021-05-14. Views should not be required to be default constructible.

<https://wg21.link/p2325r3>

[P2328R1] Tim Song. 2021-05-08. `join_view` should join all views of ranges.

<https://wg21.link/p2328r1>

[P2415R2] Barry Revzin, Tim Song. 2021-10-15. What is a view?

<https://wg21.link/p2415r2>

[P2432R1] Nicolai Josuttis. 2021-09-24. Fix `istream_view`, Rev 1.

<https://wg21.link/p2432r1>

[P2602R2] Barry Revzin. 2022-11-07. Poison Pills are Too Toxic.

<https://wg21.link/p2602r2>

[P2609R3] John Eivind Helset. 2023-02-10. Relaxing Ranges Just A Smidge.

<https://wg21.link/p2609r3>

[P2997R1] Barry Revzin, Tim Song. 2024-03-22. Removing the common reference requirement from the indirectly invocable concepts.

<https://wg21.link/p2997r1>

[P3329R0] Nicolai Josuttis. 2024-11-13. Healing the C++ Filter View.

<https://wg21.link/p3329r0>

[P3431R0] Jonathan Müller. 2025-01-13. Deprecate const-qualifier on begin/end of views.

<https://wg21.link/p3431r0>

1. Far from the first one.↵
2. Incidentally, [P3329R0] also totally misunderstands what “non-modifying” means in this context (and even does so in **red text** as if it’s a gotcha of some sort). It means the call expression does not “modify” its inputs, as that term is defined in 18.2 [concepts.equality]. But there’s no rule that you have to understand what is being changed before proposing changes.↵
3. What does it even *mean* to, for example, “find the first occurrence of the needle in the haystack” if the needle (or the haystack) can spontaneously mutate whenever the algorithm looks at it?↵
4. More like maim. Perhaps an [Auchenai Soulpriest](#) is in play?↵
5. If it is allowed to be called multiple times, that is. begin can only be called once on non-forward ranges.↵
6. Ironically, this is an attempt to make views behave more like containers. [P3329R0] repeatedly complains that views are not like containers in various aspects, yet where the library actually tries to make them more similar where it can, it attracted even more vehement complaints. No good deed goes unpunished, I suppose.↵
7. It’s not that simple, either, because you don’t want to pay the caching cost when adapting a range that *does* have constant-time begin. So we’d need a whole new set of concepts and opt-ins for this to work well. As usual, there isn’t even the faintest hint of any of this in the paper.↵
8. As of 2025-02-09. By comparison, range-v3 has 4185 stars on that date.↵
9. Including more than ten reflector threads since 2022, all started by the same individual.↵